

Lecture Note on most frequently used graphs (basic) in Stata: YRF 2024 Class 5.

Author: Md Johirul Islam

Created on 2024/10/08

Types of Graphs (mostly used)

We will cover the following graphs:

1. **Bar graph (horizontal and vertical):** mostly used to show the frequency or relative frequency of a categorical variable
2. **Line or connected graph:** Mostly used to show trend
3. **K-Density:** To show the distribution of a continuous variable.
4. **Coeff-plot:** To show the regression coefficients and standard errors.

Setting up the environment and colors

First install necessary scheme for the graph. Here I use schemepack by Asjad Naqvi. To install this, type `ssc install schemepack`.

Then set the color as follows:

```
global skyblue "86 180 233"
global blue "0 114 178"
global teal "17 222 245"
global orange "213 94 0"
global green "0 158 115"
global yellow "230 159 0"
global purple "204 121 167"
global lavender "154 121 204"
global cherry "200 0 0"
global tangerine "255 86 29"
global peach "251 162 127"
global blueberry "64 105 166"
global slate "106 90 205"
global peank "219 112 147"
```

Now, we are good to go.

Horizontal barplot

Suppose we want to see the learning levels of children (%) in Bangla by urban-rural and school type.

First load the data:

```
use "prepare_data_example", clear
```

Then do some necessary data cleaning to create the graph as follows:

```
tab Bangla, gen(b)
    tab English, gen(e)
    tab Math, gen(m)
    recode b1-m5 (1=100)

    clonevar str=strata
    la def str 0 "U" 1 "R", modify
    la val str str

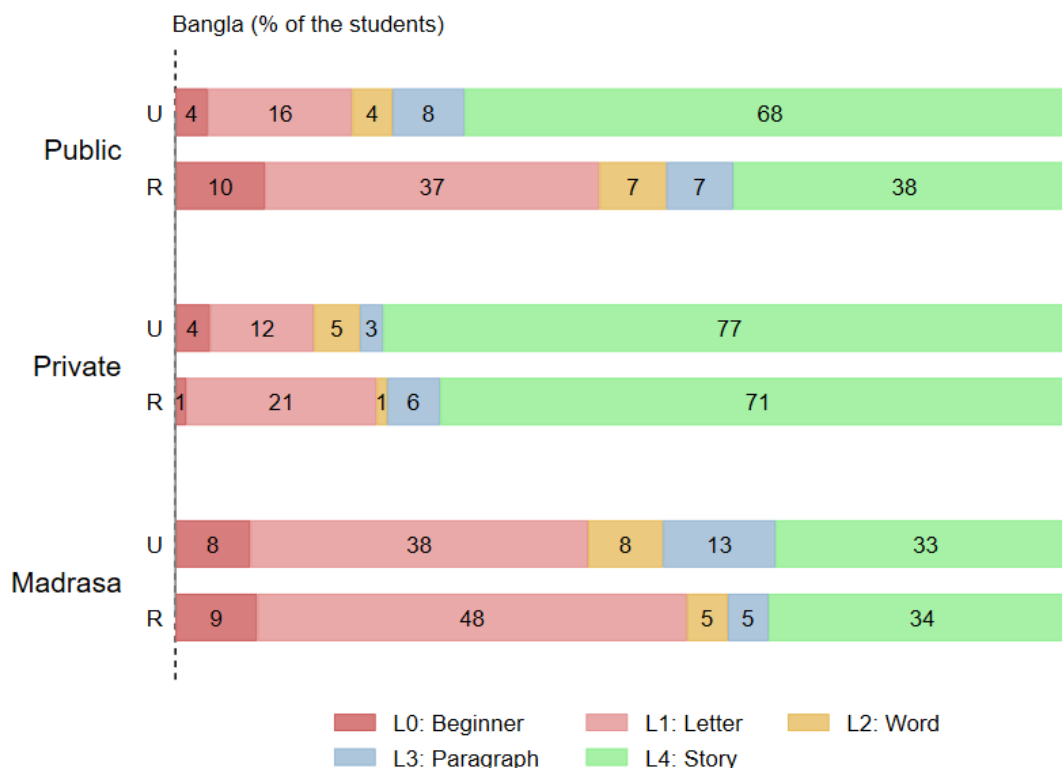
    clonevar sch=sch_type
    la def sch 1 "Public" 2 "Private" 3 "Madrasa", modify
    la val sch sch
    clonevar m11=m1
    replace m11=. if sch==2 & str==0
```

Then then the code is

```
graph hbar b1 b2 b3 b4 b5, over(str, gap(5)) ///
    over(sch, gap(55)) ///
    stack asyvars legend (label(1 "L0: Beginner") label(2 "L1:
Letter") label(3 "L2: Word") label(4 "L3: Paragraph") label(5 "L4: Story")
row(2) size(small)) ///
    ysca() ylabel(none) yline(0, lcolor(black)) ///
    blabel(bar, format(%4.0f) size(small)color (black) ///
    position(center)) bar(1, bcolor("$l0")) bar(2,
bcolor("$l1")) bar(3, bcolor("$l2")) bar(4, bcolor("$l3")) ///
    bar(5, bcolor("$l4")) ///
    legend(cols(1) row(2) pos(6) size(small)) ///
    title("Bangla", justification(left) margin(b+1 t-1 l-1)
bexpand size(small) ///
    color (black)) ytitle("", size(small)) ///
    note("",size(small)) plotregion(fcolor(white)) ///
    graphregion(fcolor(white) margin(r=8 l=8)) ///
    name(blg1, replace) scheme(white_tableau)

graph export "horizontal_bar.png", replace
```

which produces the following graph:



Beautiful graph, right?? But let's know what each option plays role in making this complete graph.

Options of Graph

The given Stata code is used to create a stacked horizontal bar graph using the `graph hbar` command. The `over(str, gap(5))` option displays the bars grouped by the `str` variable with a gap of 5 between them, while `over(sch, gap(55))` adds a further grouping by the `sch` variable with a larger gap of 55. The `stack` option combines the bars into a single bar per group, while `asyvars` assigns each category a different color. The `legend` option customizes the legend, specifying labels for each stack level (L0 to L4), arranging them in two rows, and using a small font size. The `ysca()` option controls the scaling of the y-axis, `ylabel(none)` removes labels from the y-axis, and `ylines(0, lcolor(black))` adds a black horizontal line at `y=0`. The `blabel()` option displays data labels inside the bars, formatted as integers, using a small font size and centered in black. Colors for each bar are set with `bar(1, bcolor("$l0"))` through `bar(5, bcolor("$l4"))`. The `title()` option adds a left-aligned title "Bangla" with small size, black text, and specified margins. `yttitle()` removes the title for the y-axis. The `note()` option is left blank but specifies a small font size for any notes. `plotregion(fcolor(white))` and `graphregion(fcolor(white) margin(r=8 l=8))` set the background color of the plot and graph regions to white, adjusting the margins on the right and left sides. Finally, `name(blg1, replace)` assigns the graph a name and allows it to be replaced if it already exists, while `scheme(white_tableau)` applies a specific style to the graph. Finally, `graph export` saves the graph in your desire format.

Line or Connected graph

Let's prepare the variables

```
levelsof bang_5, local(ban5)
foreach m of local ban5 {
    gen bang_5_`m' = 100*`m'.bang_5
}

levelsof eng_6, local(eng6)
foreach c of local eng6 {
    gen eng_6_`c' = 100*`c'.eng_6
}

levelsof math_5, local(math5)
foreach c of local math5 {
    gen math_5_`c' = 100*`c'.math_5
}
```

What did the above code do actually?

The provided Stata code scales scores for Bangla, English, and Math by a factor of 100 and creates new variables for each unique level. The command `levelsof bang_5, local(ban5)` stores unique values of `bang_5` in `ban5`, and the `foreach` loop generates new variables `bang_5_m'` using `gen bang_5_m' = 100*m'.bang_5` for each level. This process repeats for `eng_6` and `math_5`, using `local(eng6)` and `local(math5)`, respectively, creating scaled variables like `eng_6_c'` and `math_5_c'`. The new variables represent the original values multiplied by 100 for easier analysis.

Let's draw the plot!

Now the code is:

```
preserve

collapse (mean) bang_5_* eng_6_* math_5_* (first) gender, by(edu_level)
drop if edu_level ==.

twoway (connected bang_5_1 edu_level, lcolor("$darkgreen")
mcolor("$darkgreen")) ///
      (connected eng_6_1 edu_level, lcolor(red) mcolor(red)) ///
      (connected math_5_1 edu_level, lcolor(purple)
mcolor(purple)) ///
      , yline(20000) ///
      xlabel(0 "Pre-Primary" 1 "Class 1" 2 "Class 2" 3"Class 3"
4"Class 4" 5 "Class 5" 6 "Class 6" 7 "Class 7" 8 "Class 8" 9 "Class 9-10" 10
"Class 11-12", angle(45) labsize(vsmall)) ///
      xtitle("", size()) ///
```

```

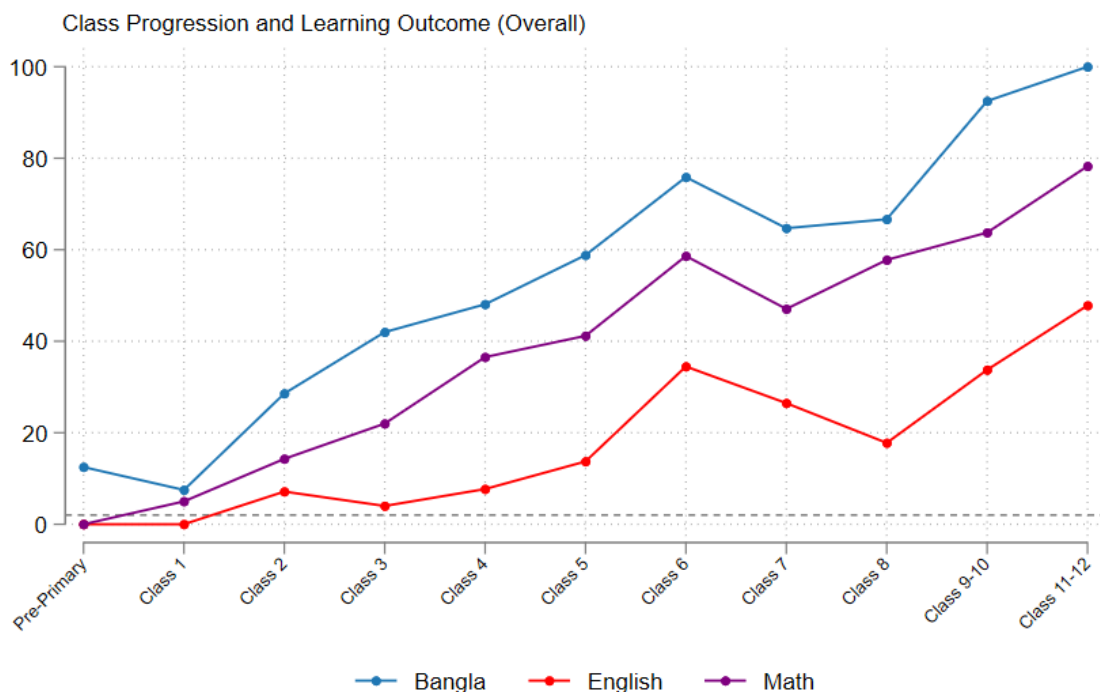
        legend (label(1 "Bangla") label(2 "English") label(3
"Math") row(1) cols() pos(bottom) size(small)) ///
        title("Class Progression and Learning Outcome (Overall)",
justification(left) margin(b+1 t-1 l-1) bexpand size(small)) ///
        ytitle("", size(small)) ///
        note("",size(small)) plotregion(fcolor(white)) ///
        graphregion(margin(r = 8 l = 8 b = 8 t = 8)) ///
        scheme(white_tableau)

graph export "l4_grade_progression.png", replace

restore

```

which renders this:



Breakdown on the codes

The provided Stata code generates a line plot to visualize learning outcomes across different education levels for three subjects: Bangla, English, and Math. The first part of the script focuses on preparing the data for visualization. The `collapse (mean) bang_5_* eng_6_* math_5_* (first) gender, by(edu_level)` command is used to calculate the mean values of variables beginning with `bang_5_`, `eng_6_`, and `math_5_` for each unique `edu_level`. Additionally, this command keeps the first value encountered for the `gender` variable within each group of

edu_level. This is followed by the drop if edu_level == . command, which removes any observations with missing values for edu_level, ensuring a clean dataset for the graph.

The graph creation itself is handled by the twoway command, which is used to plot a combined line graph containing three connected plots. Each line represents one of the subjects: Bangla, English, or Math. The command `(connected bang_5_1 edu_level, lcolor("$darkgreen") mcolor("$darkgreen"))` plots the mean Bangla scores by education level, using a dark green color for the line and markers. Similarly, `(connected eng_6_1 edu_level, lcolor(red) mcolor(red))` plots English scores using red for both the line and markers, and `(connected math_5_1 edu_level, lcolor(purple) mcolor(purple))` displays the Math scores with a purple line and markers. The `yline(20000)` option adds a horizontal line at y=20000, which can serve as a visual reference point for interpreting the data.

For the x-axis, `xlabel(...)` defines labels corresponding to each education level, ranging from “Pre-Primary” to “Class 11-12.” The labels are displayed at a 45-degree angle, using a very small font size for clarity. The `xtitle("", size())` command specifies that there should be no title for the x-axis. To distinguish between the different lines, the `legend(...)` option customizes the legend with labels for each subject (“Bangla,” “English,” and “Math”), arranges them in a single row, places the legend at the bottom of the graph, and adjusts the font size to small. Additionally, the `title(“Class Progression and Learning Outcome (Overall)”, justification(left) margin(b+1 t-1 l-1) bexpand size(small))` sets a title for the graph, adjusting the position and using a small font size for a neat appearance.

The graph also includes adjustments to the overall presentation. The `ytile(“”, size(small))` command indicates that there will be no title for the y-axis. The `note(“”, size(small))` command reserves space for notes without displaying any text, maintaining a clean look. The commands `plotregion(fcolor(white))` and `graphregion(margin(r = 8 l = 8 b = 8 t = 8))` ensure that the background of the graph remains white and adjust the margins for a balanced display. The `scheme(white_tableau)` option is used to apply a specific graphical style that matches the overall design theme of the graph.

Finally, the graph is exported using the `graph export` “l4_grade_progression.png”, replace command. This saves the graph as a PNG file named l4_grade_progression.png, replacing any existing file with the same name. The result is a color-coded, easy-to-read line graph that visually represents the progression of average learning outcomes across different education levels for the three subjects, with a design tailored for clarity and precision.

Vertical barplot

```
**# Load data
    use "dfs_sample_data.dta", clear

    clonevar savings = f1
    recode savings (1=100)
    egen tot_savings = rowtotal(f2 f3 f4)
```

```

gen debt = 1 if f7==1 | f9 ==1 | f11==1
replace debt = 0 if debt !=1 & f7 !=.

clonevar debt_1 = debt
recode debt_1 (1=100)

egen tot_debt = rowtotal(f8 f10 f12)

```

Breakdown on the codes

The Stata code creates new variables and modifies existing ones related to savings and debt. First, a new variable savings is created as a duplicate of an existing variable f1, and its value of 1 is replaced with 100. Then, the total savings is calculated by summing the values from three variables (f2, f3, f4) into a new variable called tot_savings.

For debt, a new variable debt is generated and assigned the value 1 (indicating “yes”) if any of the variables f7, f9, or f11 equal 1. If none of these are true and f7 is not missing, debt is changed to 0 (indicating “no”). The debt variable is then duplicated into debt_1, and its value of 1 is recoded as 100. Finally, the total debt is calculated by summing three variables (f8, f10, f12) into a new variable tot_debt.

Let’s draw the graph

```

*debt and savings (%)
graph bar savings debt_1, bargap(80) ///
asyvars legend (label(1 "Savings") label(2 "Debt") label(3 "") label(4
"")) label(5 "") row() size(small)) ///
ysca(range(0, 100)) ylabel(#5) yline(0, lcolor(black)) ///
blabel(bar, format(%4.0f) size(small) color (black) ///
position(outside)) bar(1, bcolor("$yellow")) bar(2, bcolor("$qual2"))
///
legend(cols(2) row(1) pos(6) size()) ///
title("Savings and Debt (% of the respondents)", justification(left)
margin(b+1 t-1 l-1) bexpand size(small) ///
color (black)) ytitle("", size(small)) ///
note("",size(medium)) plotregion(fcolor(white)) ///
name(debtsav_1, replace) scheme(white_tableau) ///
graphregion(margin( r = 38 l = 38 t = 7 b = 7))

*avg debt vs avg. savings
graph bar (mean) tot_savings tot_debt, bargap(80) ///
asyvars legend (label(1 "Savings") label(2 "Debt") label(3 "") label(4
"")) label(5 "") row() size(small)) ///
ysca(range(0, 100)) ylabel(#5) yline(0, lcolor(black)) ///
blabel(bar, format(%4.0f) size(small) color (black) ///
position(outside)) bar(1, bcolor("$yellow")) bar(2, bcolor("$qual2"))
///
legend(cols(1) row(1) pos(6) size()) ///
title("Avg. Savings vs Avg. Debt (in taka)", justification(left)
margin(b+3 t+5 l-1) bexpand size(small) ///

```

```

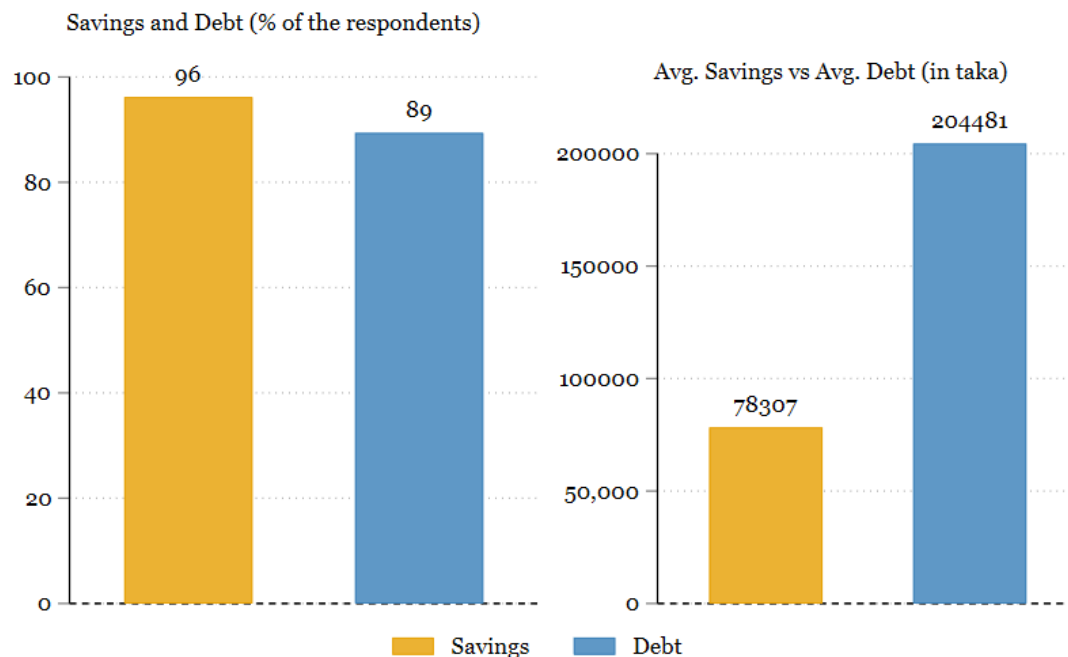
color (black)) ytitle("", size(small)) ///
note("",size(medium)) plotregion(fcolor(white)) ///
name(debtsav_2, replace) scheme(white_tableau) ///
graphregion(margin( r = 38 l = 38 t = 7 b = 7))

grc1leg debtsav_1 debtsav_2, rows(1) iscale(1) legendfrom(debtsav_2)
xcommon cols(2) ysize(5) xsize(5) imargin(0 0 0 0) graphregion(margin(r=10
l=10))

graph export "vertical_graph_combined.png", replace

```

The commands above produce the following graph:



Breakdown on the codes

The Stata code consists of two key sections that create bar charts for debt and savings comparisons, followed by combining the graphs into one display. Finally, the combined graph is exported as an image.

First, the code generates a bar graph displaying the percentage of respondents who report savings (savings) or debt (debt_1). The bars are spaced with a gap of 80%, and a legend is added to label the first bar as "Saving" and the second as "Debt." The y-axis is scaled between 0 and 100, with labels shown at intervals of 5, and a horizontal line at 0 for reference. The bars are labeled with their values in black, placed outside the bars. The first bar (savings) is colored yellow, and the

second bar (debt) uses a predefined color (“\$qual2”). The graph is titled “Savings and Debt (% of the respondents)” and has a clean layout with a white background and custom margins.

The second part of the code generates a similar bar chart, but this time it displays the average savings (tot_savings) and average debt (tot_debt). It follows the same formatting as the first graph, including the colors, labels, and title “Avg. Savings vs Avg. Debt (in taka).”

Finally, the `grc1leg` command combines these two graphs (debtsav_1 and debtsav_2) into a single image with a shared legend and common x-axis. The graphs are displayed side by side in one row, with adjusted margins and size. The final graph is then exported as an image file called “vertical_graph_combined.png.”

In summary, this code creates two bar charts—one showing the percentage of respondents with savings or debt, and the other comparing average savings and debt amounts—before combining them into one image for easy comparison.

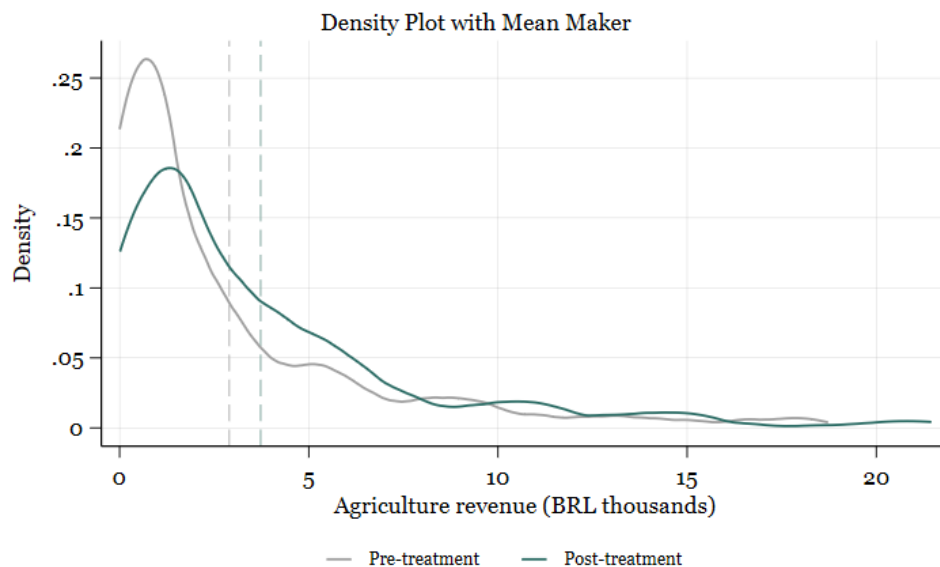
K-density graphs

```
*load data
use "https://github.com/worldbank/stata-visual-
library/raw/master/Library/data/density-av.dta", clear

*store mean values
sum      revenue if post == 0
local    pre_mean = r(mean)
sum      revenue if post == 1
local    post_mean = r(mean)

*draw the graph
twoway   (kdensity revenue if post == 0, color(gs10)) ///
         (kdensity revenue if post == 1, color(emerald)) ///
, ///
  xline(`pre_mean', lcolor(gs12) lpattern(dash)) ///
  xline(`post_mean', lcolor(eltgreen) lpattern(dash)) ///
  legend(order(1 "Pre-treatment" 2 "Post-treatment")) ///
  xtitle(Agriculture revenue (BRL thousands)) ///
  ytitle(Density) ///
  bgcolor (white) graphregion(color(white)) ///
  scheme(cblind1) ///
  graphregion(margin( r = 18 l = 18 t = 15 b = 15)) ///
  title("Density Plot with Mean Maker", justification(center)
color(black) span pos(17))
```

The commands above produce the following graph:



Breakdown on the codes

This Stata code loads a dataset, calculates the mean values of revenue before and after a specific event (treatment), and draws a density plot to visualize the distribution of revenues for both the pre-treatment and post-treatment periods, with the mean values marked by vertical lines.

Loading the data: The code starts by loading a dataset from an external source using the `use` command. This clears any previous data in Stata's memory and loads the file "density-av.dta".

Storing mean values: Next, the code calculates and stores the mean values of the revenue variable for both pre-treatment (`post == 0`) and post-treatment (`post == 1`) periods. The `sum` (summary) command calculates summary statistics, and the `local` command stores the calculated mean in a local macro (`pre_mean` for pre-treatment and `post_mean` for post-treatment).

Drawing the density plot: The `twoway` command is used to create a kernel density plot, which shows the distribution (density) of the revenue variable for two groups: pre-treatment and post-treatment. The `kdensity` function plots the density for revenue where `post == 0` (pre-treatment) in light gray (`color(gs10)`) and for revenue where `post == 1` (post-treatment) in emerald green (`color(emerald)`).

Two vertical lines are added to indicate the mean values of revenue for both groups: one for the pre-treatment mean (`pre_mean`), drawn in dashed gray (`lcolor(gs12) lpattern(dash)`), and one for the post-treatment mean (`post_mean`), drawn in dashed green (`lcolor(eltgreen) lpattern(dash)`).

The graph has a legend that labels the two lines as “Pre-treatment” and “Post-treatment,” and titles for both the x-axis (agriculture revenue in BRL thousands) and y-axis (density). The background of the graph is set to white (bgcolor(white)), and a colorblind-friendly scheme (cblind1) is applied. The margins around the graph are customized, and a centered title “Density Plot with Mean Marker” is added in black.

In summary, this code creates a density plot to visually compare the distribution of agricultural revenue before and after treatment, marking the average (mean) values of revenue for both periods.

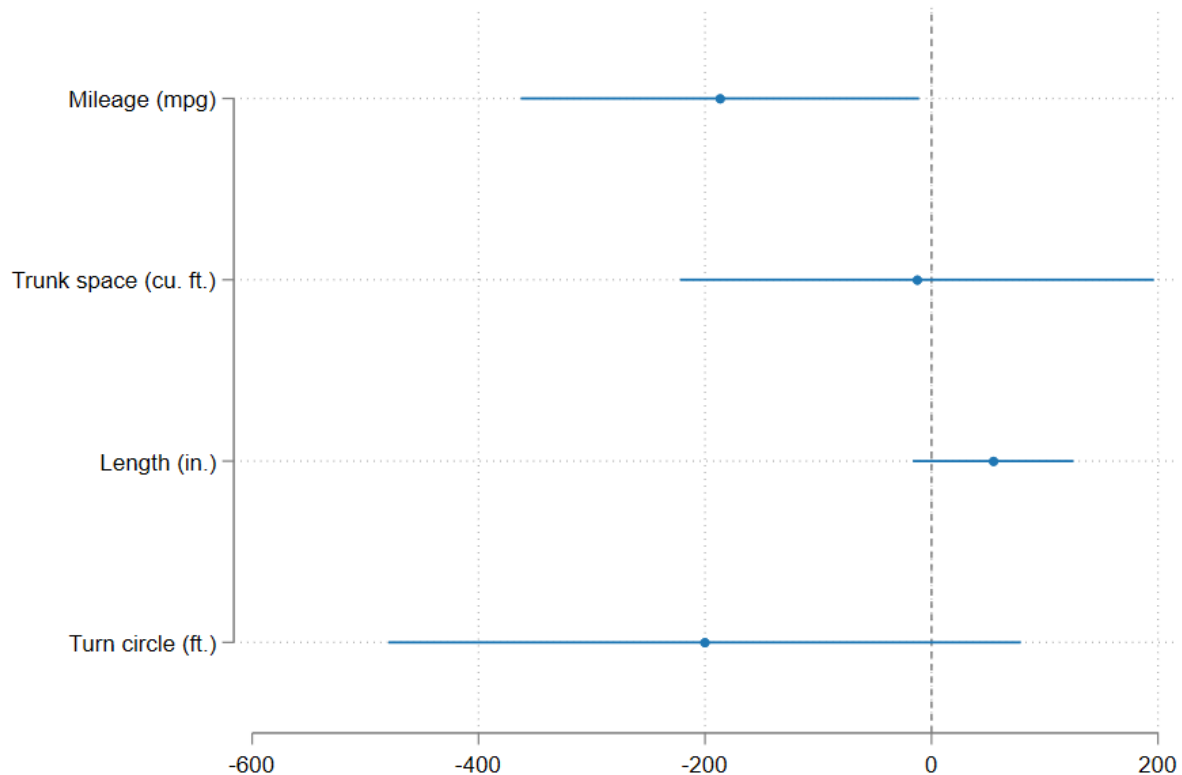
Coeff-plot

Horizontal coeff plot

```
*default is horizontal
sysuse auto, clear
regress price mpg trunk length turn
coefplot, drop(_cons) xline(0) scheme(white_tableau)

graph export "coeff1.png", replace
```

which gives:



Breakdown on the codes

This Stata code performs a regression analysis and then creates a coefficient plot based on the results of the regression. Here's a breakdown:

1. Loading the dataset: The code begins by loading the built-in auto dataset using `sysuse auto, clear`. This dataset contains information about different car models, including their prices, fuel efficiency, and physical dimensions.
2. Regression analysis: The `regress` command runs an ordinary least squares (OLS) regression with the dependent variable price (car price) and independent variables mpg (miles per gallon), trunk (trunk size), length (car length), and turn (turning circle). This regression estimates the relationship between these variables and car price.
3. Coefficient plot: The `coefplot` command generates a plot of the regression coefficients (excluding the constant term `_cons`). This plot visualizes the estimated effect of each variable (mpg, trunk, length, and turn) on price, with a vertical line at 0 (set by `xline(0)`) to help see which coefficients are significantly different from zero. The plot uses the `white_tableau` color scheme for a clean look.
4. Exporting the graph: Finally, the graph is saved as a PNG image with the filename "coeff1.png" using the `graph export` command, and the `replace` option ensures that if an image with the same name exists, it will be overwritten.

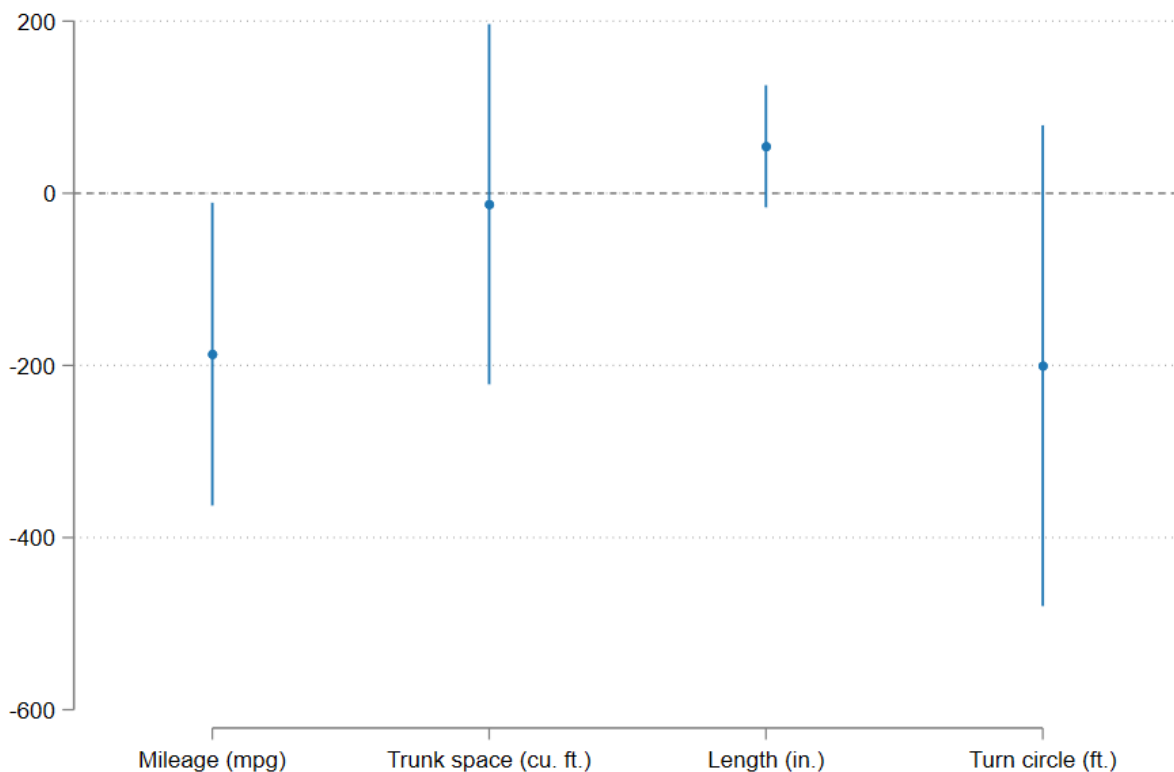
In summary, this code runs a regression to see how different car attributes affect the price and visualizes the results with a coefficient plot, which is then exported as an image.

Vertical Coeff-plot

Just add `vertical` in graph option and change `xline(0)` to `yline(0)`. So, here is the full code

```
coefplot, vertical drop(_cons) yline(0) scheme(white_tableau)  
graph export "coeff2.png", replace
```

which gives:

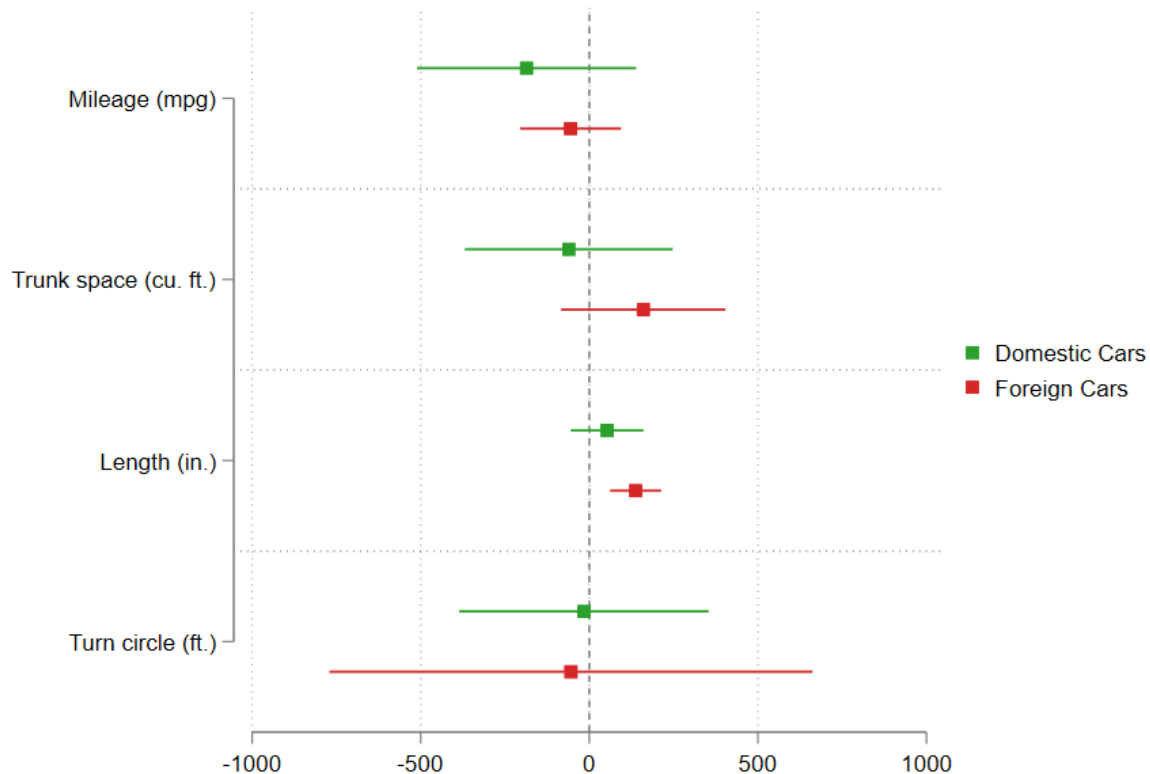


By group: heterogeneity

```
regress price mpg trunk length turn if foreign==0  
estimates store D  
regress price mpg trunk length turn if foreign==1  
estimates store F  
  
coefplot D F, drop(_cons) xline(0) msymbol(S) ///  
p1(label(Domestic Cars) pstyle(p3))          ///  
p2(label(Foreign Cars) pstyle(p4))          ///  
scheme(white_tableau)
```

```
graph export "coeff3.png", replace
```

which gives:



Breakdown on the codes

This Stata code performs a regression analysis separately for domestic and foreign cars, stores the results, and then creates a coefficient plot to compare the regression coefficients. Here's a breakdown of each step:

1. **Regression for Domestic Cars:** The code first runs a regression analysis using the `regress` command for domestic cars only (where `foreign == 0`). It uses `mpg` (miles per gallon), `trunk` (trunk size), `length` (car length), and `turn` (turning radius) as independent variables, with `price` as the dependent variable. The results are stored under the name `D` using the `estimates store D` command.
2. **Regression for Foreign Cars:** Next, a similar regression is run for foreign cars (where `foreign == 1`). The same independent variables are used, and the results are stored under the name `F` with the command `estimates store F`.
3. **Coefficient Plot:** The `coefplot` command creates a plot comparing the coefficients from the two regression results (`D` for domestic cars and `F` for foreign cars). The `drop(_cons)` option excludes the constant term from the plot. The `xline(0)` option adds a vertical line at

0 to help visualize which coefficients are statistically significant. The `msymbol(S)` option specifies the symbol used for the points on the plot. The `p1(label(Domestic Cars) pstyle(p3))` and `p2(label(Foreign Cars) pstyle(p4))` options label the two sets of points in the plot and set different styles for the domestic and foreign car coefficients. The `scheme(white_tableau)` option applies a clean color scheme to the graph.

4. Exporting the Graph: Finally, the plot is exported as a PNG image file named “coeff3.png” using the `graph export` command with the `replace` option to overwrite any existing file with the same name.

In summary, this code conducts separate regression analyses for domestic and foreign cars, compares their coefficients using a coefficient plot, and exports the resulting plot as an image.